

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0503

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 30%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.	BL3 – Apply	5
2	The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.	BL3 – Apply	5
3	A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.	BL3 – Apply	5
4	Debug the given relational algebra expression that produces incorrect output for a natural join operation.	BL3 – Apply	5
5	The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.	BL3 – Apply	5
6	Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.	BL6 – Create	5
7	The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.	BL3 – Apply	5

8	Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table.	BL3 – Apply	5
9	Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes.	BL6 – Create	5
10	Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.	BL3 – Apply	5

Code of Conduct:

*I B.Poorvaja(192473008) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained

Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

1. FIX INCORRECT JOIN SYNTAX

INCORRECT QUERY

SELECT *

FROM employee, attendance

WHERE employee.EmployeeID = attendance.EmployeeID;

OUTPUT:

```
mysql> SELECT *
-> FROM employee, attendance
-> WHERE employee.EmployeeID = attendance.EmployeeID;
```

EmployeeID	EmployeeName	Department	AttendanceID	EmployeeID	AttendanceDate	Status
101	Rahul	IT	1	101	2026-08-01	Present
101	Rahul	IT	2	101	2026-08-02	Present
102	Sneha	HR	3	102	2026-08-01	Absent
102	Sneha	HR	4	102	2026-08-02	Present
103	Amit	Finance	5	103	2026-08-01	Present
103	Amit	Finance	6	103	2026-08-02	Present

6 rows in set (0.00 sec)

CORRECTED QUERY:

SELECT DISTINCT

employee.EmployeeID,

employee.EmployeeName,

employee.Department,

attendance.AttendanceDate,

attendance.Status

FROM employee

INNER JOIN attendance

ON employee.EmployeeID = attendance.EmployeeID;

OUTPUT:

```
mysql> SELECT DISTINCT
-> employee.EmployeeID,
-> employee.EmployeeName,
-> employee.Department,
-> attendance.AttendanceDate,
-> attendance.Status
-> FROM employee
-> INNER JOIN attendance
-> ON employee.EmployeeID = attendance.EmployeeID;
```

EmployeeID	EmployeeName	Department	AttendanceDate	Status
101	Rahul	IT	2026-08-01	Present
101	Rahul	IT	2026-08-02	Present
102	Sneha	HR	2026-08-01	Absent
102	Sneha	HR	2026-08-02	Present
103	Amit	Finance	2026-08-01	Present
103	Amit	Finance	2026-08-02	Present

2. MISSING CORRELATION

ERROR QUERY

```
SELECT EmployeeName
FROM employee
WHERE EmployeeID IN
(
    SELECT EmployeeID
    FROM performance
    WHERE Rating >
    (
        SELECT AVG(Rating)
        FROM performance
    )
);
```

OUTPUT:

```
mysql> SELECT EmployeeName
-> FROM employee
-> WHERE EmployeeID IN
-> (
->     SELECT EmployeeID
->     FROM performance
->     WHERE Rating >
->     (
->         SELECT AVG(Rating)
->         FROM performance
->     )
-> );
+-----+
| EmployeeName |
+-----+
| Rahul        |
| Amit         |
+-----+
2 rows in set (0.00 sec)
```

CORRECT QUERY:

```
SELECT EmployeeName
FROM employee e
WHERE EmployeeID IN
(
    SELECT EmployeeID
    FROM performance
    WHERE Rating >
    (
        SELECT AVG(Rating)
        FROM performance
        WHERE EmployeeID = e.EmployeeID
    )
);
```

OUTPUT:

```
FROM employee e
WHERE EmployeeID IN
(
    SELECT EmployeeID
    ' at line 1
mysql> SELECT EmployeeName
-> FROM employee e
-> WHERE EmployeeID IN
-> (
->     SELECT EmployeeID
->     FROM performance
->     WHERE Rating >
->     (
->         SELECT AVG(Rating)
->         FROM performance
->         WHERE EmployeeID = e.EmployeeID
->     )
-> );
Empty set (0.00 sec)

mysql> SELECT * FROM performance;
+-----+-----+-----+-----+
| PerformanceID | EmployeeID | Rating | ReviewMonth |
+-----+-----+-----+-----+
| 1 | 101 | 5 | August |
| 2 | 102 | 4 | August |
| 3 | 103 | 5 | August |
+-----+-----+-----+-----+
```

3.

ERROR QUERY:

```
CREATE VIEW Employee_View AS
SELECT *
FROM Employee
WHERE Dept_ID = Department.Dept_ID;
```

OUTPUT:

```
mysql> CREATE VIEW Employee_View AS
-> SELECT *
-> FROM Employee
-> WHERE Dept_ID = Department.Dept_ID;
ERROR 1054 (42S22): Unknown column 'Dept_ID' in 'where clause'
mysql> |
```

CORRECT QUERY:

```
CREATE VIEW Employee_View AS
SELECT
    EmployeeID,
    EmployeeName,
    Department
FROM Employee;
```

OUTPUT:

```
mysql> CREATE VIEW Employee_View AS
-> SELECT *
-> FROM Employee
-> WHERE Dept_ID = Department.Dept_ID;
ERROR 1054 (42S22): Unknown column 'Dept_ID' in 'where clause'
mysql> CREATE VIEW Employee_View AS
-> SELECT
->     EmployeeID,
->     EmployeeName,
->     Department
-> FROM Employee;
ERROR 1050 (42S01): Table 'Employee_View' already exists
mysql> SELECT * FROM Employee_View;
+-----+-----+-----+
| EmployeeID | EmployeeName | Department |
+-----+-----+-----+
| 101 | Rahul | IT |
| 102 | Sneha | HR |
| 103 | Amit | Finance |
+-----+-----+-----+
```

4.

INCORRECT RELATIONAL ALGEBRA EXPRESSION

Employee ⋈ Department

CORRECT RELATIONAL ALGEBRA EXPRESSION

Employee ⋈ Employee.DepartmentID = Department.DepartmentID Department

QUERY:

SELECT E.EmployeeID, E.EmployeeName, D.DepartmentName

FROM Employee E

JOIN Department D

ON E.DepartmentID = D.DepartmentID;

OUTPUT:

EmployeeID	EmployeeName	DepartmentID	DepartmentName
101	Rahul	D1	IT
102	Sneha	D2	HR
103	Amit	D3	Finance

5.

INCORRECT QUERY

```
CREATE TABLE Department_Q5 (  
    DepartmentID INT PRIMARY KEY,  
    DepartmentName VARCHAR(50)  
);
```

```
CREATE TABLE Employee_Q5 (  
    EmployeeID INT PRIMARY KEY,  
    EmployeeName VARCHAR(50),  
    DepartmentID INT,  
    FOREIGN KEY (DepartmentID)  
    REFERENCES Department_Q5(DepartmentID)  
);
```

```
INSERT INTO Employee_Q5  
VALUES (101,'Rahul',10);
```

OUTPUT:

```
mysql> INSERT INTO Employee_Q5  
-> VALUES (101,'Rahul',10);  
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`hrdb`.`employee_q5`, CONSTRAINT `employee_q5_ibfk_1` FOREIGN KEY  
DepartmentID) REFERENCES `department_q5` (`DepartmentID`))
```

CORRECTED QUERY:

```
INSERT INTO Department_Q5
```

```
VALUES (10,'IT');
```

```
INSERT INTO Employee_Q5
```

```
VALUES (101,'Rahul',10);
```

```
SELECT * FROM Employee_Q5;
```

OUTPUT:

```
mysql> SELECT * FROM Employee_Q5;
+-----+-----+-----+
| EmployeeID | EmployeeName | DepartmentID |
+-----+-----+-----+
|          101 | Rahul          |           10 |
+-----+-----+-----+
1 row in set (0.00 sec)
```

6.

INCORRECT QUERY:

```
SELECT EmployeeName
FROM Employee
WHERE Department = (
    SELECT DepartmentName
    FROM Department
    WHERE DepartmentID = 'D1'
);
```

OUTPUT:

```
mysql> SELECT EmployeeName
-> FROM Employee
-> WHERE Department = (
->     SELECT DepartmentName
->     FROM Department
->     WHERE DepartmentID = 'D1'
-> );
+-----+
| EmployeeName |
+-----+
| Rahul        |
+-----+
```

CORRECTED QUERY:

```
SELECT E.EmployeeName
FROM Employee E
INNER JOIN Department D
ON E.Department = D.DepartmentName
WHERE D.DepartmentID = 'D1';
```

OUTPUT:

```
mysql> SELECT E.EmployeeName
-> FROM Employee E
-> INNER JOIN Department D
-> ON E.Department = D.DepartmentName
-> WHERE D.DepartmentID = 'D1';
+-----+
| EmployeeName |
+-----+
| Rahul        |
+-----+
1 row in set (0.00 sec)
```

7.

INCORRECT QUERY:

```
SELECT Department, EmployeeName, COUNT(*)
FROM Employee
GROUP BY Department;
```

INCORRECT QUERY OUTPUT:

```
mysql> SELECT Department, EmployeeName, COUNT(*)
-> FROM Employee
-> GROUP BY Department;
ERROR 1055 (42000): Expression #2 of SELECT list is not in GROUP BY clause and contains nonaggregated column 'hrdb.Employee.EmployeeName' which is not functionally dependent on columns in GROUP BY clause; this is incompatible with sql_mode=only_full_group_by
```

CORRECTED QUERY:

```
SELECT Department, COUNT(*) AS TotalEmployees
FROM Employee
GROUP BY Department;
```

CORRECTED QUERY OUTPUT:

```
mysql> SELECT Department, COUNT(*) AS TotalEmployees
-> FROM Employee
-> GROUP BY Department;
+-----+-----+
| Department | TotalEmployees |
+-----+-----+
| IT         | 1              |
| HR         | 1              |
| Finance    | 1              |
| 10         | 1              |
+-----+-----+
4 rows in set (0.00 sec)
```

8.

INCORRECT QUERY

```
UPDATE Employee
SET Department = 'IT';
```

OUTPUT:

```
mysql> SELECT * FROM Employee;
+-----+-----+-----+
| EmployeeID | EmployeeName | Department |
+-----+-----+-----+
| 101        | Rahul        | IT         |
| 102        | Sneha        | IT         |
| 103        | Amit         | IT         |
| 104        | Kiran        | IT         |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

CORRECTED QUERY

```
UPDATE Employee SET Department = 'HR' WHERE EmployeeID = 102;
UPDATE Employee SET Department = 'Finance' WHERE EmployeeID = 103;
UPDATE Employee
SET Department = 'IT'
WHERE EmployeeID = 101;
SELECT * FROM Employee;
```

OUTPUT;


```
mysql> SELECT * FROM Employee;
+-----+-----+-----+
| EmployeeID | EmployeeName | Department |
+-----+-----+-----+
| 101 | Rahul | IT |
| 102 | Sneha | HR |
| 103 | Amit | Finance |
| 104 | Kiran | Sales |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

9.

INCORRECT QUERY:

```
SELECT EmployeeName
FROM Employee
WHERE Department = (
    SELECT DepartmentName
    FROM Department
    WHERE DepartmentID = 'D1'
);
```

OUTPUT:

```
mysql> SELECT EmployeeName
-> FROM Employee
-> WHERE Department = (
->     SELECT DepartmentName
->     FROM Department
->     WHERE DepartmentID = 'D1'
-> );
+-----+
| EmployeeName |
+-----+
| Rahul |
+-----+
1 row in set (0.00 sec)
```

CORRECTED QUERY:

```
SELECT E.EmployeeName
FROM Employee E
INNER JOIN Department D
ON E.Department = D.DepartmentName
WHERE D.DepartmentID = 'D1';
```

OUTPUT:

```
mysql> SELECT E.EmployeeName
-> FROM Employee E
-> INNER JOIN Department D
-> ON E.Department = D.DepartmentName
-> WHERE D.DepartmentID = 'D1';
+-----+
| EmployeeName |
+-----+
| Rahul |
+-----+
1 row in set (0.00 sec)
```

10.

INCORRECT EXPRESSION (TUPLE RELATIONAL CALCULUS)

$$\{ E.EmployeeName \mid Employee(E) \wedge \forall D (Department(D) \rightarrow E.Department = D.DepartmentName) \}$$

OUTPUT:

Empty Result Set

CORRECTED EXPRESSION:

$\{ E.EmployeeName \mid Employee(E) \wedge \exists D (Department(D) \wedge E.Department = D.DepartmentName) \}$

OUTPUT:

```
mysql> SELECT E.EmployeeName
-> FROM Employee E
-> JOIN Department D
-> ON E.Department = D.DepartmentName;
+-----+
| EmployeeName |
+-----+
| Rahul        |
| Sneha        |
| Amit         |
+-----+
```